

Unidad 4: Entrega continua (CI/CD)

Esta sección marca el paso final en la profesionalización de un repositorio: la transformación del código en un producto entregable y confiable. Tras haber organizado el trabajo en equipo y definido las reglas de colaboración, el siguiente paso es automatizar la vigilancia y el despliegue de nuestra infraestructura de software.

En esta unidad, aprenderás a **Diseñar pipelines de Integración Continua (CI)**, estableciendo "estaciones de inspección" automáticas que validan cada línea de código mediante pruebas, análisis de seguridad y construcción de artefactos antes de permitir cualquier fusión. Abordaremos la lógica del **Versionado Semántico (SemVer)** como el lenguaje universal para comunicar la magnitud de nuestros cambios, asegurando que tanto usuarios como sistemas dependientes sepan qué esperar de cada nueva versión. Finalmente, exploraremos la **Entrega y Despliegue Continuo (CD)**, donde orquestaremos el movimiento del código a través de distintos entornos (Desarrollo, Pruebas y Producción) e implementaremos mecanismos de resiliencia como el **Rollback**, garantizando que el sistema pueda recuperarse instantáneamente ante cualquier imprevisto en el mundo real.

1. Integración Continua (CI): El Filtro de Calidad Automático

La CI responde a una pregunta crítica: **"¿Este nuevo cambio rompe algo en el proyecto?"**. En lugar de esperar a una fase de pruebas manuales al final del mes, la CI ejecuta una batería de pruebas cada vez que un desarrollador sube código.

1.1 El Concepto de Pipeline y la Filosofía "Fail-Fast"

Un **Pipeline** es la ejecución secuencial de pasos automatizados. La regla de oro es el **Fail-Fast (Fallo Rápido)**: si el código falla en el paso 1 (estilo de código), no tiene sentido gastar tiempo y dinero ejecutando el paso 4 (pruebas costosas). El proceso se detiene, se notifica al desarrollador y se bloquea la integración.



1.2 Anatomía Detallada de un Pipeline (Stages)

Un pipeline profesional organiza sus tareas en **Stages** (Etapas). Cada etapa debe ser exitosa para pasar a la siguiente:

1. **Checkout (Obtención del código):** El servidor de CI crea una máquina virtual o un contenedor temporal y clona el repositorio. Esto asegura que las pruebas se hagan sobre el código real y no sobre "lo que funciona en mi PC".
2. **Environment Setup (Preparación):** Se configura el entorno exacto. Si el proyecto usa Node.js v20.x, el robot instala esa versión y descarga todas las librerías necesarias. Esto garantiza **repetibilidad**: el resultado será el mismo hoy y en un año.
3. **Lint & Static Analysis (Limpieza):** Antes de probar si el código funciona, se comprueba si está bien escrito. Los *Linters* buscan variables no usadas o errores de formato, mientras que el *Análisis Estático* busca patrones de código peligrosos.
4. **Unit Tests (Pruebas Unitarias):** Es el corazón de la CI. Se ejecutan miles de pequeñas pruebas que verifican la lógica individual de cada función. Si el 100% no pasa, el código no se integra.
5. **Build (Construcción):** En lenguajes que requieren compilación (como Java, C# o TypeScript), el robot genera los archivos ejecutables. Si hay un error de sintaxis que el desarrollador no vio, el *Build* fallará aquí.
6. **Security Scan (Auditoría):** Se analizan las dependencias del proyecto. Si usas una librería que tiene una vulnerabilidad de seguridad reportada, el pipeline te avisará (o detendrá el proceso) para evitar subir código vulnerable a producción.

1.3 El Archivo de Configuración: El "Cerebro" del Pipeline



En la era moderna, usamos **Pipeline as Code**. Esto significa que las instrucciones del robot están escritas en un archivo de texto (YAML) dentro del propio repositorio. Esto permite que el pipeline evolucione junto con el código.

Si usamos **GitHub Actions**, el archivo reside en `.github/workflows/main.yml` y se estructura en tres pilares:

- **Events (Triggers):** Son los disparadores. Puedes configurar el pipeline para que se active solo cuando hay un `push` a la rama `main`, o cada vez que alguien abre un `Pull Request`.
- **Jobs (Trabajos):** Son bloques de ejecución. Un pipeline puede tener un job para "Pruebas" y otro para "Seguridad". Los jobs pueden correr en paralelo para que el pipeline termine más rápido.
- **Steps (Pasos):** Son los comandos de terminal individuales. Por ejemplo: `run: npm test`.

  Consideración de Rendimiento: En estrategias como **Trunk-based Development**, donde los desarrolladores integran cambios varias veces al día, el pipeline debe ser optimizado (usando *cache* de dependencias) para que los resultados lleguen en pocos minutos. Un pipeline lento destruye la agilidad del equipo.

1.4 Resultados y Visibilidad

El éxito de la CI reside en el **feedback inmediato**. El desarrollador recibe una señal verde o roja directamente en su Pull Request.

- **Verde:** El código es seguro para ser revisado por un humano.
- **Rojo:** El desarrollador debe corregir el error localmente y volver a subirlo antes de que nadie pierda tiempo revisando un código que ni siquiera compila.

2. Versionado Semántico (SemVer): La Identidad del Software

En un entorno automatizado, los números de versión no son aleatorios; son un sistema de comunicación técnica. El **Versionado Semántico** utiliza el formato **X.Y.Z** (Mayor.Menor.Parche) para informar el impacto de los cambios.

- **MAJOR (X.0.0):** Se incrementa cuando realizas cambios que **rompen la compatibilidad**. El usuario de tu código tendrá que hacer ajustes para que su sistema siga funcionando.
- **MINOR (0.Y.0):** Se incrementa cuando añades **funcionalidades** de manera compatible con lo anterior. Es una evolución segura.
- **PATCH (0.0.Z):** Se incrementa para **correcciones de errores** menores que no alteran el funcionamiento esperado.

El rol de los Tags: En Git, cada versión SemVer se materializa mediante un **Tag**. El Pipeline de CD suele "escuchar" la creación de estos Tags para decidir si debe iniciar un despliegue a producción.

3. Entrega y Despliegue Continuo (CD)

Si la **CI** garantiza la integridad técnica del código, la **CD (Continuous Delivery / Deployment)** se encarga de la logística: transportar ese código de forma segura y predecible hacia el usuario final. En esta etapa, la prioridad es reducir el riesgo mediante la automatización y la segregación de entornos.

3.1 Entrega vs. Despliegue: El control del último paso

La diferencia fundamental radica en el nivel de automatización y quién tiene la autoridad para el paso final a producción:

- **Entrega Continua (Continuous Delivery):** El código siempre está en un estado "listo para desplegar". Tras superar la CI, el proceso se detiene antes de llegar a producción, esperando una **aprobación humana** manual. Es ideal para organizaciones donde el negocio decide el momento estratégico de salida.



- **Despliegue Continuo (Continuous Deployment):** El flujo es 100% automático. Si las pruebas pasan, el código llega a producción sin intervención humana. Requiere una madurez técnica elevada y una confianza total en la batería de tests.

3.2 Despliegue (Deployment) vs. Lanzamiento (Release)

Para un profesional de ingeniería, estos términos no son sinónimos:

- **Despliegue:** Es un evento técnico. Consiste en colocar el código en los servidores y ponerlo a ejecutar.
- **Lanzamiento:** Es un evento de negocio. Consiste en habilitar la funcionalidad para los usuarios.

Mediante técnicas como las **Feature Flags**, un equipo puede *desplegar* código constantemente a producción sin *lanzar* la funcionalidad hasta que el producto esté listo. Esto permite separar el riesgo técnico del impacto comercial.

3.3 La Gestión de Entornos e Inmutabilidad

El código viaja a través de estaciones de control que minimizan el impacto de posibles errores. Un principio fundamental aquí es el de **Artefactos Inmutables**: el paquete que se prueba en Staging debe ser el mismo que se despliega en Producción, sin volver a compilarlo.

1. **Entorno de Desarrollo (Dev):** Espacio dinámico donde los desarrolladores integran y prueban cambios diarios.
2. **Entorno de Staging / QA:** Una réplica exacta de Producción. Aquí se realizan pruebas de usuario (UAT), de seguridad y de estrés sobre el artefacto final.
3. **Entorno de Producción (Prod):** El entorno real donde residen los usuarios y los datos críticos.

4. Ejemplo Práctico: GitHub Actions (CI + CD)



Para entender cómo estas dos fases conviven, analizamos un archivo de configuración `.github/workflows/main.yml`. Este "robot" hará dos trabajos: validar (CI) y desplegar (CD).

```
name: Pipeline de Ingeniería

on:
  push:
    branches: [ main ]  # Se dispara al subir a main
  pull_request:
    branches: [ main ]  # Se dispara al proponer cambios a main

jobs:
  # FASE DE CI (INTEGRACIÓN CONTINUA)
  continuos-integration:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3     # 1. Obtiene el código
      - name: Setup Node.js
        uses: actions/setup-node@v3   # 2. Prepara el entorno
        with:
          node-version: '20'
      - run: npm install              # 3. Instala dependencias
      - run: npm run lint             # 4. Limpieza (Análisis estático)
      - run: npm test                 # 5. Pruebas unitarias

  # FASE DE CD (ENTREGA CONTINUA)
  continuous-delivery:
     needs: continuos-integration    # Solo corre si la CI fue exitosa
(Fail-Fast)
     if: github.event_name == 'push' # Solo despliega si es un cambio
definitivo
    runs-on: ubuntu-latest
    steps:
```




```
- name: Deploy to Staging
  run: echo "Desplegando en el servidor de pruebas..."
# Aquí iría la conexión por SSH o hacia servicios como AWS/Azure
```

5. Resiliencia y Mitigación: El Rollback

Incluso con pipelines de CI/CD impecables, el software puede fallar en el mundo real debido a imprevistos en la infraestructura o comportamientos inesperados de los usuarios. La **resiliencia** no es la ausencia de errores, sino la capacidad técnica de recuperarse de ellos en el menor tiempo posible.

5.1 El Concepto de Rollback (Marcha atrás)

Un **Rollback** consiste en revertir el sistema a una versión estable anterior de manera casi instantánea al detectar una anomalía en producción.

- **A nivel de Git:** Se identifica el **Tag (SemVer)** de la versión anterior que era sólida y se utiliza como referencia para el despliegue.
- **A nivel de Infraestructura y Artefactos:** Para que un rollback sea efectivo, los **Artefactos deben ser inmutables**. Esto significa que no volvemos a compilar el código; simplemente "re-encendemos" el paquete (binario o contenedor) que ya había sido validado previamente. Las herramientas modernas de CD permiten que, si la nueva versión falla, el tráfico se redirija automáticamente a la versión anterior en milisegundos.

5.2 Estrategias de Mitigación de Riesgo

Para minimizar el impacto de un despliegue fallido, se utilizan estrategias que permiten probar el código en producción antes de comprometer a todos los usuarios:

- **Blue-Green Deployment:** Se mantienen dos entornos idénticos. El entorno **Blue** (versión actual) sigue activo mientras despliegas la nueva versión en el **Green**. El cambio es instantáneo mediante el balanceador de carga. Si el



entorno Green presenta errores, el rollback es tan sencillo como redirigir el tráfico de vuelta al Blue.

- **Canary Deployment:** Inspirado en los canarios de las minas, esta estrategia libera la nueva versión solo para un pequeño porcentaje de usuarios (ej. 5%). Se monitorean los logs en tiempo real; si no hay errores, se aumenta gradualmente el tráfico hasta el 100%. En caso de falla, el impacto se limita a un grupo mínimo de personas.